10

Version control

Commits, branches, and why being able to revert makes risky changes cheap

Working with an assistant means accepting large batches of changes you didn't type, and that's only safe at speed if any state of the project is recoverable. Version control is the mechanism. This chapter covers it at mental-model depth — what a commit is, what the recovery moves are, and the working agreements that keep generated changes disposable. Command fluency isn't the goal: your assistant runs the commands, and you decide the moves.

PART

The model

Commits

A **commit** is a snapshot of the whole project at one moment, with a message saying what changed and why. The project's **history** is the chain of these snapshots. A commit differs from a saved file in what it enables: you can return to any snapshot, compare any two, and see exactly what changed between them — which makes every past state of the project a place you can stand on again. If you've never made a commit on purpose, you may still have them — a tool may have been committing as it worked — and the first ask is whether the project has version control at all and where its history lives; chapter 0's stack prompt includes it.

Branches

A **branch** is a movable pointer to a commit — a named line of work. The branch called main is conventionally the line you'd deploy; work happens on other branches and gets **merged** back when it's ready. The value at recognition depth: a branch makes an experiment free-standing, so abandoning it costs nothing and touching main is a deliberate act.

The unit of a commit

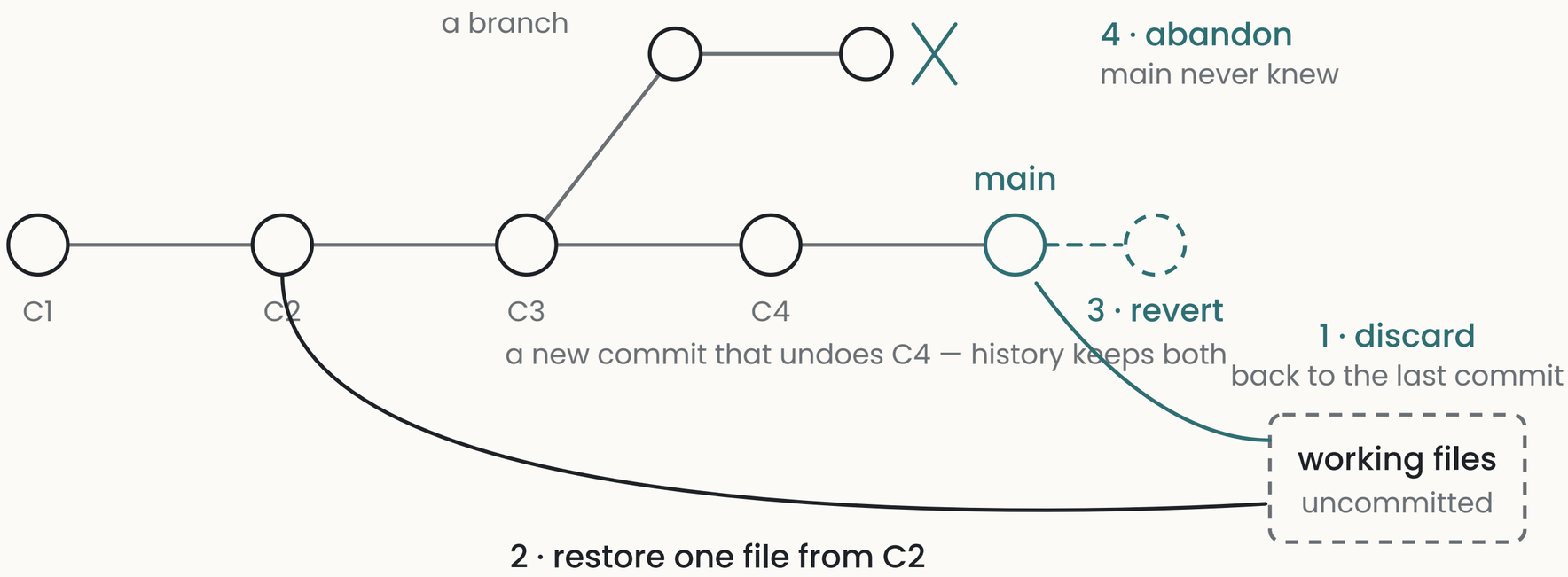
A commit should hold one concern: one feature, one fix, one rename. The reason matters more with generated code than it did before: a commit is the unit of undo, so a batch that mixes three unrelated changes can only be kept or discarded as three-at-once. Assistants produce large batches by default; asking for one commit per concern — or asking the assistant to split a mixed batch before committing — keeps each change independently revertible.

The recovery moves

Four moves cover most recoveries, and knowing their names is enough to ask for them:

1. **Discard uncommitted changes** — put the working files back to the last commit. The move for a generation that went wrong before anything was committed.
2. **Restore one file** — take a single file back to any earlier commit, leaving the rest alone.
3. **Revert a commit** — a new commit that undoes an old one. History keeps both, which is why this is the safe undo: nothing is erased, and the undo is itself recorded.
4. **Abandon a branch** — walk away from a line of work. Main never knew about it.

HISTORY, AND THE FOUR RECOVERY MOVES



Two boundaries around the safety net are worth knowing. Uncommitted work isn't in it — the net holds what was committed. And history can be rewritten by a class of operations (force-pushes, amends, squashes) that replace snapshots rather than adding them; those operations remove ground the net stood on, which is why they're a deliberate decision and never routine cleanup.

Review as verification

The **diff** — the exact difference between two snapshots — is the artifact review happens on, and it's what chapter 11's asks run over: what did this touch, what did it delete. Reading the diff rather than the summary is the practice; even working alone, the moment before committing is where surprises are cheapest.

Working agreements with an assistant

Three habits make the mechanics into a net that's actually under you:

1. **Commit before any large generation.** The commit is the place you'll return to if the generation is bad; it has to exist before the generation starts.
2. **One risky task, one branch.** The experiment stays free-standing until it earns a merge.
3. **History rewrites are yours to authorize.** An assistant may run version control commands; replacing history isn't cleanup it should ever do unasked.

PART

What goes wrong

1 Generation on top of uncommitted work

a bad result that can't be discarded without taking good work with it.

ASK "Before you start: what's uncommitted right now? Commit it or set it aside, then name the commit we'd return to if this goes wrong."

CHECK the named commit exists before the generation begins.

TELL *a large generation started while unrelated changes sit uncommitted.*

2 History rewritten as cleanup

snapshots replaced, and the net shortened, without a decision.

ASK "List every operation this session that rewrote or removed history. Say none if none."

CHECK commits you noted earlier still exist under the same identifiers.

TELL *force-pushes, amended or squashed commits appearing in a session that wasn't asked for them.*

THE DIRECT TEST

Before accepting any large change, say — out loud or in the session — which commit you'd return to if it's wrong. If you can't name one — including because you've never looked — ask the assistant to name it and show it to you; if it can't, the safety net isn't under this change, and that's the moment to fix it. Practicing the return once on a harmless change turns the answer from a belief into a rehearsed move.

PART

Going deeper

These prompts are for your assistant, and each comes with a note on what a good response looks like.

D1 · Read me my history

GROUNDED EXPLANATION

Show me my project's history for the last month in plain language: for each commit, what changed and why, judging from its message and its actual diff. Flag any commit where the message and the diff disagree, and any commit that mixes unrelated concerns.

WHAT A GOOD RESPONSE LOOKS LIKE

A good response reads like a diary of the project. The flags are the findings — a message-diff disagreement is a summary that didn't match the change, which chapter 11 treats as a shape worth knowing.

D2 · The revert drill

BUILD A TOY

Help me practice the recovery moves on a harmless change: make a small deliberate mistake, commit it, then revert it; then restore one file to how it was three commits ago; then start a branch, make a change on it, and abandon it. Narrate what each move did to the history.

WHAT A GOOD RESPONSE LOOKS LIKE

In about half an hour, all four moves become things you've done. The narration matters — ask again whenever a move's effect on history isn't clear.

D3 · The recoverability audit

AUDIT

Answer for my project as it stands right now: if the working files were lost, what would survive? If the laptop were lost? List what's uncommitted, what's committed but not pushed anywhere, and whether the project exists anywhere beyond this machine. Say gone explicitly for anything that would be gone.

WHAT A GOOD RESPONSE LOOKS LIKE

A good response is three short lists and their consequences. "Committed but only on this machine" is the common surprise, and it means the net is only as safe as the laptop.

D4 · The mess quiz

PREDICTION QUIZ

Quiz me on recoveries. One at a time, describe a mess — a bad change just generated over my half-finished work, a wrong commit already merged to main, a good file deleted four commits ago — and ask me which recovery move applies. Wait for my answer, then correct me with the reason.

WHAT A GOOD RESPONSE LOOKS LIKE

Commit before each correction. The mixed-uncommitted-work case is the one the first entry exists to prevent; getting it wrong in the quiz is cheaper than in the repository.

WHERE THIS CONNECTS

Chapter 11 · The loop is the working cycle these moves make safe — its restart advice assumes the net is under you. **Chapter 5 · Production** covers rolling back a deploy, which is the same idea one layer up: code reverts change history, deploy rollbacks change what's running.